

1. WAP to construct a binary tree using array and traverse the binary tree.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
int *tree = NULL;
int size;

void set_root(int val){
    tree[0] = val;
}

void set_left_child(int val, int parent){
    tree[(2*parent)+1] = val;
}

void set_right_child(int val, int parent){
    tree[(2*parent)+2] = val;
}

void display(){
    printf("Tree Nodes: ");
    for (int i=0; i<size; i++){
        printf("%d ", tree[i]);
    }
    printf("\n");
}

int main(){
    printf("Enter Tree Size: ");
    scanf("%d", &size);

    tree = (int *) malloc(size*sizeof(int));

    for (int i=0; i<size; i++){
        tree[i] = -1;
    }

    set_root(10);
    set_left_child(20,0);
    set_right_child(30,0);
    set_left_child(40,1);
    set_right_child(50,1);
    set_left_child(60,2);
    set_right_child(70,2);

    display();
    return 0;
}
```

Output:

```
PS D:\DS\DSL\LAB7> gcc p1.c -o p1.exe
PS D:\DS\DSL\LAB7> .\p1.exe
Enter Tree Size: 7
Tree Nodes: 10 20 30 40 50 60 70
PS D:\DS\DSL\LAB7>
```

2. WAP to construct a binary tree using linked list and write the functions for InOrder, PreOrder & PostOrder traversal.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left_child;
    struct node *right_child;
};

struct node *create(){
    int value;
    struct node *new;
    new = (struct node *) malloc(sizeof(struct node));
    scanf("%d", &value);

    if(value == -1)
        return NULL;

    new->data = value;
    printf("Left child of %d: ", value);
    new->left_child = create();
    printf("Right child of %d: ", value);
    new->right_child = create();

    return new;
}

void InOrder(struct node *ptr){
    if(ptr == NULL){
        return;
    }
    InOrder(ptr->left_child);
    printf("%d ", ptr->data);
    InOrder(ptr->right_child);
}

void PreOrder(struct node *ptr){
    if(ptr == NULL){
        return;
    }
    printf("%d ", ptr->data);
    PreOrder(ptr->left_child);
    PreOrder(ptr->right_child);
}

void PostOrder(struct node *ptr){
    if(ptr == NULL){
        return;
    }
}
```

```

        PostOrder(ptr->left_child);
        PostOrder(ptr->right_child);
        printf("%d ", ptr->data);
    }

int main(){
    struct node *root;
    printf("For No Node -> Enter -1\n");
    printf("Enter Root Node: ");
    root = create();
    printf("InOrder Traversal: ");
    InOrder(root);
    printf("\n");
    printf("PreOrder Traversal: ");
    PreOrder(root);
    printf("\n");
    printf("PostOrder Traversal: ");
    PostOrder(root);
    printf("\n");
    return 0;
}

```

Output:

```

PS D:\DS\DSL\LAB7> gcc p2.c -o p2.exe
PS D:\DS\DSL\LAB7> .\p2.exe
For No Node -> Enter -1
Enter Root Node: 10
Left child of 10: 20
Left child of 20: 40
Left child of 40: -1
Right child of 40: -1
Right child of 20: 50
Left child of 50: -1
Right child of 50: -1
Right child of 10: 30
Left child of 30: -1
Right child of 30: -1
InOrder Traversal: 40 20 50 10 30
PreOrder Traversal: 10 20 40 50 30
PostOrder Traversal: 40 50 20 30 10
PS D:\DS\DSL\LAB7>

```

3. WAP to count the number of leaf nodes present in a binary tree by linked list.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left_child;
    struct node *right_child;
};

struct node *create(){
    int value;
    struct node *new;
    new = (struct node *) malloc(sizeof(struct node));
    scanf("%d", &value);

    if(value == -1)
        return NULL;

    new->data = value;
    printf("Left child of %d: ", value);
    new->left_child = create();
    printf("Right child of %d: ", value);
    new->right_child = create();
    return new;
}

int count_leaf(struct node *ptr){
    if(ptr == NULL)
        return 0;
    if(ptr->left_child == NULL && ptr->right_child == NULL)
        return 1;
    return count_leaf(ptr->left_child) + count_leaf(ptr->right_child);
}

int main(){
    struct node *root;
    int count = 0;
    printf("For No Node -> Enter -1\n");
    printf("Enter Root Node: ");
    root = create();
    count = count_leaf(root);
    printf("Total Leaf Nodes = %d\n", count);
    return 0;
}
```

Output:

```
PS D:\DS\DSL\LAB7> gcc p3.c -o p3.exe
PS D:\DS\DSL\LAB7> .\p3.exe
For No Node -> Enter -1
Enter Root Node: 10
Left child of 10: 20
Left child of 20: 40
Left child of 40: -1
Right child of 40: -1
Right child of 20: 50
Left child of 50: -1
Right child of 50: -1
Right child of 10: 30
Left child of 30: -1
Right child of 30: -1
Total Leaf Nodes = 3
PS D:\DS\DSL\LAB7>
```